

How Text Becomes Numbers in NLP

Tokenization, IDs, Embeddings, Context Windows

Introduction “How Letters Become Words & How Tokens Represent All Languages”

Introduction

- English has **only 26 letters**.
- Yet the language contains **hundreds of thousands to millions of words**.
- Large Language Models (LLMs) also rely on a **small finite set of tokens** (vocabulary).
- These tokens can represent **all English words** *and* all global languages.

English Alphabet: Only 26 Letters

- Lowercase: 26
- Uppercase: 26
- Total alphabetic symbols: **52**
- Additional characters (punctuation, digits): ~40–50 commonly used

Yet English speakers: - *Understand*: **20,000–35,000 words** - *Use actively*:
~**3,000–5,000 words** - *Dictionaries*: **400k–600k words** - *Full English
lexicon*: **1–2+ million words** including: - technical - scientific - proper
nouns - archaic terms

How Do 26 Letters Produce Millions of Words?

Combination & Morphology

Key Principles

- **Concatenation:**
run + ning, un + break + able
- **Prefixes/Suffixes:**
un-, re-, pre-, anti-, -ing, -tion, -ize
- **Compound words:**
football, underestimate, pickled-ball
- **Borrowed words:**
from French, Latin, Greek, German, etc.

From Letters → Subwords → Words

LLMs do *not* operate on full words or letters.
They use **subwords** and **tokens**.

Examples

Word	Subwords (tokens)
international	inter + national
unbelievable	un + believe + able
running	run + ning
picklebally	pickle + ball + y

Subwords allow LLMs to: - Represent **any English word** - Represent **misspellings** - Represent **newly invented words** - Represent **proper names** - Represent **infinite linguistic combinations**

Why LLMs Don't Need Huge Vocabularies

- English has millions of potential words
- But LLMs use **vocabulary sizes between 8k and 200k**

Why so small?

- **Subwords combine** to form entire words
- Rare words get **split into multiple tokens**
- Common words get **1 token**
- All text becomes a sequence of token IDs

This allows LLMs to: - Work across **all languages** - Keep models **compact** - Train efficiently - Handle **infinite possible words**

Illustration: Subword Coverage

Even a tokenizer with **8,000 tokens** can encode:

- all English dictionary words
- all slang
- all misspellings
- all proper nouns
- all scientific words
- all made-up words
- all numbers
- all emojis
- all names in all languages

because rare words are broken apart:

internationalization → inter + national + ization

Vocabulary Sizes of Major LLMs

GPT Family

- **GPT-2 / GPT-3 / GPT-NeoX**: ~**50,000** tokens
- **GPT-4**: estimated ~**200,000** tokens
- **GPT-4o**: ~**200,000** tokens (multilingual)

LLaMA Family

- **LLaMA-1**: 32,000
- **LLaMA-2**: 32,000
- **LLaMA-3 / 3.1**: **128,256** (very large)

Mistral / Mixtral

- All models: **32,000**

Gemma (Google)

- **256,000** tokens (largest among major LLMs)

Why Bigger Vocabularies Help

A larger vocab allows:

- fewer splits for rare words
- better multilingual coverage
- better code + math handling
- more direct whole-word tokens
- shorter sequences → faster inference

But:

- vocabulary too large = sparse training
- training becomes harder
- embedding matrix too big

Most open-source LLMs choose 32k tokens
because it's the best trade-off.

Summary

- English has **26 letters**, but produces **millions of words**.
- LLMs use **subwords and tokens** (not words or letters).
- A tokenizer with **8k–200k tokens** can represent:
 - all English words
 - all human languages
 - numbers, emojis, URLs, names
 - infinite invented words
- Major LLMs vary from **32k vocab** (LLaMA, Mistral) to **200k+ vocab** (GPT-4o, Gemma, Qwen).

Thank You

Questions?

Why Convert Text to Numbers?

Neural networks operate only on **numbers**.

Text is **symbolic** and must be encoded numerically.

Pipeline:

Text $\rightarrow$ Tokens $\rightarrow$ IDs $\rightarrow$ Embeddings $\rightarrow$ Tensors

Predictive Modeling Analogy

Traditional ML:

$$(X_1, X_2, \dots, X_k) \rightarrow y$$

Text modeling:

$$(\text{token}_{i-2}, \text{token}_{i-1}) \rightarrow \text{token}_i$$

What Are Embeddings?

- Embeddings convert **words/tokens into numbers** so that machine learning models can process text.
- Two major types:
 - **Static embeddings** (fixed, context-independent)
 - **Dynamic embeddings** (learned, context-sensitive, used in LLMs)

Definition

- A **static embedding** assigns **one fixed vector** to each word.
- The vector does **not change with context**.
- Examples include:
 - Bag-of-Words (BoW)
 - TF-IDF
 - Word2Vec
 - GloVe

Static Embeddings Example

TF-IDF Example

Given the corpus:

- 1 “The cat sat on the mat”
- 2 “The dog sat on the rug”

TF-IDF assigns values such as:

Word	TF-IDF
cat	0.52
dog	0.52
the	0.00
sat	0.11

These numbers are **static** and do not change with context.

Static Embeddings Example

Word2Vec Example

Word2Vec learns one vector per word such as:

cat = [0.12, -0.89, 0.22]

dog = [0.15, -0.91, 0.25]

- Same vector for “cat” in:
 - “The **cat** sat on the mat”
 - “The **cat** is not cute”
- No context awareness.

Definition

- **Dynamic embeddings** are generated by deep neural networks (RNNs, LSTMs, Transformers).
- They **change depending on context**.
- Used in modern LLMs:
 - GPT
 - BERT
 - LLaMA
 - Mistral

Dynamic Embeddings Example

Transformer Example

Token: **“bank”**

Sentence A:

“I deposited money at the **bank**.”

Sentence B:

“The river **bank** overflowed.”

Dynamic embeddings produced by the model:

cat in sentence 1 = [1.12, -0.33, 2.55]

cat in sentence 2 = [0.90, -1.11, 2.01]

Context changes the vector.

Why Dynamic Embeddings Are Powerful

- Capture **semantic meaning**
- Capture **syntax**
- Capture **context**
- Learn deep relationships
- Improve performance in:
 - translation
 - reasoning
 - summarization
 - question answering

How Dynamic Embeddings Work

- 1 Token $\rightarrow$ token ID
- 2 Token ID $\rightarrow$ **lookup in embedding matrix** (initially random)
- 3 Add **positional embedding**
- 4 Pass through Transformer layers
- 5 Generate **contextual embedding**
- 6 Predict next token using softmax

Static vs Dynamic Embeddings: Summary

Feature	Static	Dynamic
How created?	Formula or simple model	Learned via deep training
Changes with context?	No	Yes
One vector per word?	Yes	No
Used in modern LLMs?	No	Yes
Captures semantics?	Limited	Excellent
Examples	TF-IDF, Word2Vec	GPT, BERT, LLaMA

Static Embeddings

- “cat” $\rightarrow$ same vector everywhere
- “dog” $\rightarrow$ same vector
- Does not understand meaning differences across sentences

Dynamic Embeddings

- “cat” in sentence about pets $\rightarrow$ vector A
- “cat” in sentence about danger $\rightarrow$ vector B
- Context reshapes meaning

Real Example (Toy)

Static (Word2Vec-like):

cat = [0.12, -0.89, 0.22]

dog = [0.15, -0.91, 0.25]

Dynamic (Transformer):

cat in sentence 1 = [1.12, -0.33, 2.55]

cat in sentence 2 = [0.90, -1.11, 2.01]

Closing Summary

- **Static embeddings** → one fixed vector per word
- **Dynamic embeddings** → vectors change with context, learned during training
- Dynamic embeddings make LLMs powerful for:
 - reasoning
 - translation
 - summarization
 - text generation

Tokenization

Tokenization

Tokenization splits text into:

- Word tokens
- Subword tokens (BPE)
- Characters

Models do **not** understand words — they understand **token IDs**.

Example Sentence

Sentence:

I love large language models.

Tokens:

- I
- love
- large
- language
- models
- .

Vocabulary to Token IDs

Example token ID assignments:

I $\rightarrow$ 40

love $\rightarrow$ 3047

large $\rightarrow$ 4410

language $\rightarrow$ 6439

models $\rightarrow$ 7015

. $\rightarrow$ 13

Encoded sentence:

[40, 3047, 4410, 6439, 7015, 13]

Context Window (Size = 2)

Training samples:

$[I, \textit{love}] \rightarrow \textit{large}$

$[\textit{love}, \textit{large}] \rightarrow \textit{language}$

$[\textit{large}, \textit{language}] \rightarrow \textit{models}$

$[\textit{language}, \textit{models}] \rightarrow .$

Context Window (Size = 2)

In IDs:

$[40, 3047] \rightarrow 4410$

$[3047, 4410] \rightarrow 6439$

$[4410, 6439] \rightarrow 7015$

$[6439, 7015] \rightarrow 13$

Embedding Layer (Dimension = 3)

Example embeddings:

$$40 \rightarrow (0.1, 0.2, 0.3)$$

$$3047 \rightarrow (-0.5, 0.1, 0.0)$$

$$4410 \rightarrow (0.7, -0.2, 0.4)$$

$$6439 \rightarrow (0.0, 0.3, 0.9)$$

$$7015 \rightarrow (0.4, 0.4, 0.4)$$

$$13 \rightarrow (-0.2, 0.0, 0.1)$$

Input Tensor (Context = 2)

Tokens:

[40, 3047]

Embeddings:

$$\begin{pmatrix} 0.1 & 0.2 & 0.3 \\ -0.5 & 0.1 & 0.0 \end{pmatrix}$$

Tensor shape: 2×3

Training Examples

$$[40, 3047] \rightarrow \begin{pmatrix} 0.1 & 0.2 & 0.3 \\ -0.5 & 0.1 & 0.0 \end{pmatrix} \rightarrow 4410$$

$$[3047, 4410] \rightarrow \begin{pmatrix} -0.5 & 0.1 & 0.0 \\ 0.7 & -0.2 & 0.4 \end{pmatrix} \rightarrow 6439$$

$$[4410, 6439] \rightarrow \begin{pmatrix} 0.7 & -0.2 & 0.4 \\ 0.0 & 0.3 & 0.9 \end{pmatrix} \rightarrow 7015$$

$$[6439, 7015] \rightarrow \begin{pmatrix} 0.0 & 0.3 & 0.9 \\ 0.4 & 0.4 & 0.4 \end{pmatrix} \rightarrow 13$$

Next Token Prediction

Model learns:

(2×3) tensor $\rightarrow$ Probability distribution

Example output:

$$P(\text{large}) = 0.72, P(\text{language}) = 0.10, P(\text{models}) = 0.08$$

Prediction = $\arg \max$.

Regression Analogy

NLP Concept	ML Concept
Previous tokens	Input X
Next token	Output y
Token IDs	Encoded categories
Embeddings	Learned features
Context window	Number of predictors
Sequence model	Function $f(X)$

Our Tiny Dataset

We used:

- 1 sentence
- 6 tokens
- vocab size = 6
- context = 2
- embedding dim = 3
- only 4 training pairs

Modern LLMs train on:

- Trillions of tokens
- Many languages
- Code, math, academic papers
- Social media
- Multimodal data (images, audio, video)

- 2–10 trillion tokens
- Web, code, books
- Embedding dim: $\sim 10,000$
- Context: 128k–1M
- Multimodal

- Trillions of tokens
- Strong multimodal training
- Embedding dim: 8k–16k
- Context window: up to 1M tokens

Tiny Example vs LLMs

Feature	Example	GPT-4	Gemini	Grok
Tokens	6	trillions	trillions	trillions
Vocabulary	6	100k	100k	100k
Embedding dim	3	10k	10k	8k
Context window	2	128k–1M	1M	128k– 1M
Data size	1 sentence	internet- scale	internet- scale	internet- scale

What This Comparison Shows

- Our example is tiny
- LLMs are huge
- But the pipeline is identical:

Text → Tokens → IDs → Embeddings → Model → Next Token

Key Takeaways

- NLP = converting text to numbers
- Embeddings = learned numeric vector features
- Context = model's history window
- Our example is small
- LLMs scale the same math to trillions of tokens

Model Inferences

Model Inferences

What Is Inference?

- **Training:** model learns parameters
- **Inference:** model uses learned parameters to generate predictions
- In LLMs, inference = **predicting the next token**

LLM output = sequence of repeated next-token predictions

Next-Token Prediction

During inference, the model receives:

- Previous tokens
- Hidden states
- Attention context

Then computes:

$$P(\text{next token} = t_i \mid \text{context})$$

This produces a **probability distribution** over all tokens in the vocabulary.

Example Probability Distribution

For a given context:

“The cat sat on the”

The model might output:

Token	Probability
mat	0.42
floor	0.11
table	0.07
dog	0.02
...	...

The model does **not** output a word directly — it outputs **probabilities**.

Sampling vs. Taking the Maximum

There are two ways to turn probabilities into the next token:

1. Greedy Decoding

Choose the **highest-probability** token:

$$\text{token} = \arg \max P(t)$$

Deterministic.

Always the same output.

Sampling vs. Taking the Maximum

2. Sampling

Choose the next token by **drawing from the distribution**:

$$t \sim P(t)$$

This introduces **randomness** →
responses change each time.

This is why the same prompt can produce different answers.

Why Sampling?

- Creativity
- Diversity
- Avoiding repetition
- Producing natural-sounding text
- Preventing deterministic, robotic outputs

Without sampling, LLMs would sound **identical** every time.

Temperature

Temperature **reshapes** the probability distribution.

- Low T ($T = 0-0.3$): sharp, deterministic
- Medium T ($0.7-1.0$): balanced
- High T (>1.0): creative, chaotic

Formula:

$$P_i = \frac{\exp(\log P_i / T)}{\sum_j \exp(\log P_j / T)}$$

Top-k and Top-p Sampling

Top-k

Keep only the **k most likely** tokens.

- $k=1 \rightarrow$ greedy
- $k=10 \rightarrow$ more variety
- $k=50 \rightarrow$ creative

Top-p (Nucleus Sampling)

Choose the smallest set of tokens whose cumulative probability $\geq p$.

Example: $p = 0.9 \rightarrow$ sample from the top 90% of probability mass.

More adaptive than top-k.

Why LLM Predictions Differ

Even with the same prompt:

- Temperature randomness
- Sampling randomness
- Different random seeds
- Token-by-token randomness amplifies over time
- Small probability differences $\rightarrow$ divergence

This is normal and expected.

LLM generation is **not deterministic** unless:

- Temperature = 0
- Top-k = 1
- Top-p = 1
- Fixed random seed

Otherwise → the model samples → outputs vary.

Summary

- LLMs predict a **probability distribution**, not a word
- Inference = sampling tokens sequentially
- Variation in predictions = due to sampling
- Temperature, top-k, and top-p control randomness
- Stochastic sampling is essential for creative and natural text

